

Luiss

Libera Università Internazionale
degli Studi Sociali Guido Carli

Report Demo

Data Privacy and Security

Data Science and Management (**DASMA**)

LUISS



Introduction

Goal

Produce a written technical report analyzing a **real-world CVE** in a program of **your choice**

What you'll deliver:

- A **structured report** covering the vulnerability, its impact, and its patch
- A **working Docker demo** that reproduces the attack

Need more details? Check the Project slides.

Picking your CVE

Where to look:

- NVD - National Vulnerability Database
- Mitre CVE - official CVE list
- GitHub Security Advisories - often includes patch diffs and PoC code
- Any search engine, or whatever else has all the answers

A **good CVE** should have:

- A clear, reproducible exploit
- A patch you can actually read and understand

Avoid:

- CVEs with no public technical detail
- CVEs already taken by another student

Take your time picking the right CVE, or spend the rest of the project wishing you did!

Picking your CVE: Tips

If I were you I'd...

- Pick a CVE affecting a software I'm familiar with
- Pick a CVE from a vulnerability class covered in the course - the "vulnerability type" section of the report will basically write itself
- Pick a CVE from a vulnerability class I actually understood, not just survived
- Make sure I can reproduce it before committing - a boring CVE you can reproduce beats an exciting one you can't

What we'll see today

During this lesson we'll walk through a report analyzing two CVEs in Django 4.0:

- **CVE-2022-22818**: Cross-Site Scripting vulnerability
- **CVE-2022-28346**: SQL Injection vulnerability

We'll use it to:

- Show what a good report looks like in practice
- Highlight what works and why
- Extract the structure you should follow in your own report

N.B. These CVEs are **off-limits** for your project

Section 1: Program Description

What we expect:

- A **concise (1-2 pages) introduction** to the application your CVE affects
- Informations **relevant** to understanding the vulnerability i.e., which components of the application it affects

Common mistakes:

- Copy-pasting unfocused paragraphs from the **official documentation**
- Describing the application in full with **no connection** to the vulnerability

Rule of thumb: if someone unfamiliar with the application can read your section and understand the context of the vulnerability, you've done it right

Section 1: Program Description - Hints

Ask yourself:

- What does this application do?
- What language/framework is it written in?
- What are its main components?
- Which component is affected by the vulnerability?
- How does data flow through the relevant parts?

Example: Django Framework

What is Django?

- High-level Python web framework, free and open source
- Used by Instagram, Spotify, Mozilla, many others!
- Handles the heavy lifting of web development out of the box - routing, authentication, database interaction, templating

Why it matters for this report:

- The vulnerabilities we'll see are rooted in two specific Django components:
 - The template engine - responsible for rendering HTML pages
 - The QuerySet API - responsible for interacting with the database

CVE-2022-22818

Template Tag XSS Vulnerability



Section 2: Vulnerability Type

What we expect:

- A **self-contained explanation** of the vulnerability class your CVE belongs to (XSS, SQLi, SSRF, etc.)
 - Assume the reader knows basic web concepts but not this specific vulnerability type
 - Cover the main variants, then **focus** on the one relevant to your CVE

Common mistakes:

- Going too broad
- Not connecting back to your specific CVE at the end of the section

Rule of thumb: write it as a mini-lecture on the vulnerability class, with your CVE being a case study

Section 2: Vulnerability Type - Hints

Follow a What → Cause → Impact structure: what the vulnerability is, what causes it, what an attacker can do with it

Ask yourself:

- What vulnerability class does your CVE belong to?
- What causes this type of vulnerability?
- What can an attacker do with it?
- Which variant is relevant to your CVE?

Example: Cross-Site Scripting (XSS)

Vulnerability Description:

- **WHAT:** an web vulnerability where malicious scripts are delivered, via injection, and executed in the victim's browser
- **WHY:** it is caused by user input being embedded in a page without proper validation or encoding
- **IMPACT:** it allows attackers to steal data and perform actions by impersonating the victim

Variants:

- **Reflected:** malicious code is part of the request and immediately reflected back in the response
- **Stored:** malicious code is saved on the server (e.g. in a database) and executed when the page is loaded
- **DOM-based:** malicious code manipulates the page's DOM structure directly in the browser

Relevant to our case: CVE-2022-22818 exploits a combination of Stored and DOM-based XSS, as the malicious script is saved in the database, then executed via a vulnerable debug tag that fails to encode the page's context variables.

Section 3: The Specific CVE

What we expect:

- Identify **where** the vulnerability lives
- Explain the **mechanism**: what fails, why it fails, under what conditions
- Walk through a **concrete attack scenario** step by step
- Be **precise** e.g., vague descriptions like "the input is not sanitized" are not enough

Common mistakes:

- Describing the vulnerability in **abstract** terms
- Confusing this section with the vulnerability type section - here we're talking about **this specific CVE**, not XSS in general

Section 3: The Specific CVE - Hints

This is the **heart of your report**: this is where you prove **you truly understand** your CVE, not just that you read about it

Ask yourself:

- Where does the vulnerability live?
- What is the vulnerable function/component/parameter?
- What input or condition triggers the vulnerability?
- What fails, and why does it fail?
- What does a successful attack look like, step by step?
- What can an attacker achieve by exploiting it?

Example: CVE-2022-22818

Where is the vulnerability?

- The `{% debug %}` template tag in Django 4.0: a built-in tag that prints debugging information directly on the page
- The current context (variables shared between view and template) is printed without proper encoding

What goes wrong?

- Any context variable containing JavaScript or HTML is executed by the browser instead of displayed as text
- The `Set-Cookie` header lacks `HttpOnly` → cookies and CSRF tokens are directly accessible via JavaScript

The **attack**:

- Create a user with name `" ]> } <script>alert(document.cookie)</script>`
- This closes the open debug object representation and injects a script tag directly into the DOM
- Result: browser executes the script and exposes the session cookie

Section 4: Impact & Mitigation

What we expect:

- Assess the **real-world impact**: who is affected, under what conditions, how severely
- Describe the **patch**: what changed in the code, where, and why that change fixes the problem

Common mistakes:

- **Vague** impact statements: "it may lead to data leakage" is not enough; be specific about what data, how, and in what scenario
- Describing the patch without describing **why it fixes the issue**

Rule of thumb: if your impact section could apply to any CVE, it's not specific enough

Section 4: Impact & Mitigation - Hints

Ask yourself:

- Who is affected by this vulnerability?
- Under what conditions can it be exploited?
- What is the worst case scenario for a victim?
- What specifically changed in the patch?
- Where in the code was the fix applied?
- Why does that change actually fix the problem?

Example: Impact & Patch

Impact:

- Any context variable containing executable code is printed unencoded on the page
- An attacker can steal session cookies and CSRF tokens via `document.cookie`
- Limited to applications using `{% debug %}` outside of development (uncommon but not impossible)

The **fix** - Django 4.0.2 (February 1, 2022):

- `{% debug %}` no longer outputs anything when `DEBUG=False` in `settings.py`
- When `DEBUG=True`, all context variables are now correctly encoded before being printed

Section 5: Demo

What we expect:

- A working Docker environment that **reproduces the attack**
- **Step by step** instructions to reproduce the attack
- **Screenshots** showing the exploit in action

Common mistakes:

- **Too many** screenshots - justify the ones you use
- **Skipping steps** that "seem obvious" - what's obvious to you might not be obvious to the reader

Section 5: Demo - Hints

Ask yourself:

- Can someone reproduce your exploit by following your steps alone?
- Did you explain what is happening at each step, not just what to run?
- Does every screenshot have a purpose - does it show something that words can't?

Example: Django XSS Demo

The **vulnerable environment**:

- Django 4.0.0 running in one container,
- PostgreSQL in another,
- Docker Compose to connect them
- Two endpoints: `localhost:8000/xss` and `localhost:8000/sql`

The **attack**:

- Navigate to the XSS page and create a user with name `" ]> } <script>alert(document.cookie) </script>`
- The `{% debug %}` tag fails to encode the stored value
- The browser executes the script and exposes the session cookie in an alert box

Section 6: References

What we expect:

Cite **everything** you used:

- CVE pages,
- Official documentation,
- Security advisories,
- GitHub pages,
- Blog posts,
- ...

CVE-2022-28346

QuerySet Method SQL Injection



Example: SQL Injection

Vulnerability Description:

- **WHAT:** an injection vulnerability where malicious SQL statements are embedded in user input and executed by the database
- **WHY:** it is caused by user input being incorporated into a query without proper sanitization
- **IMPACT:** it allows attackers to read, modify, or delete database content, and in some cases compromise the entire server

Variants:

- **First-Order:** injected code is executed immediately by the Database Management System
- **Second-Order:** injected code is stored first, then executed later in another query
- **Blind:** no direct output from the database; the attacker infers information from the application's behaviour

Relevant to our case: CVE-2022-28346 exploits a First-Order SQLi in Django: user input is inserted directly into a SQL query without validation, allowing an attacker to append arbitrary SQL statements.

Example: CVE-2022-28346

Where is the vulnerability?

- Three QuerySet methods in Django 4.0: `aggregate()`, `annotate()`, and `extra()`
- These methods accept a `**kwargs` dictionary: when user input is passed as dictionary keys, those keys become column aliases in the raw SQL query without sanitization

What goes wrong?

- The column alias is inserted directly into the SQL query without any validation
- A crafted alias can break out of the expected syntax and inject arbitrary SQL statements

Example with `aggregate()`:

- Legitimate alias: `maximum_age` → `SELECT MAX(age) AS maximum_age FROM sql_user`
- Malicious alias: breaks the query and appends `INSERT INTO sql_exam (name) VALUES ('SQLi'); --`

Example: Impact & Patch

Impact:

- Full SQL injection via three widely used `` methods
- An attacker can INSERT, UPDATE, or DELETE records freely
- Damage is partially limited by database integrity constraints (but not fully)

The **fix** - Django 4.0.4 (April 11, 2022):

- A new function `test_alias_sql_injection()` validates all column aliases before query execution
- Any alias containing whitespace, quotation marks, semicolons, or SQL comments is rejected
- The patch covers all three vulnerable methods consistently

Example: The Demo

The **vulnerable** environment:

- Same Docker setup as the XSS demo: Django 4.0.0 + PostgreSQL + Docker Compose
- Endpoint: `localhost:8000/sql` - a page exposing all three vulnerable query methods directly

The **attack(s)**:

- Aggregate: malicious alias injects an INSERT statement, adding a new exam record
- Annotate: malicious alias injects a DELETE statement, removing the previously inserted record
- Extra: malicious alias injects an UPDATE statement, renaming an existing exam record

Reproducing the Vulnerability: Docker Setup

EASY: ask your trusted LLM (ChatGPT, Claude, ...) to help you out!

MEDIUM: build it yourself:

1. Find the right base image on Docker Hub: look for the exact version of the software affected by your CVE
2. Write a Dockerfile that installs the vulnerable version and any dependencies
3. Write a `docker-compose.yml` to wire up the services (e.g., app + database)
4. Add any setup scripts needed to initialize the environment (e.g., database schema, seed data)

Reproducing the Vulnerability: Dockerfile & Requirements

Dockerfile:

```
# Base image – use the Python version required by the vulnerable app
FROM python:3

# Set the working directory inside the container
WORKDIR /code

# Install dependencies
COPY requirements.txt /code/
RUN pip install -r requirements.txt

# Copy the rest of the application code
COPY . /code/
```

requirements.txt:

```
Django==4.0 #the vulnerable version
psycopg2
```


Reproducing the Vulnerability: docker-compose

```
version: "3.9"

services:
  db:  # Use the database your app depends on
    image: postgres
    volumes:  # Persist database data between container restarts
      - ./data/db:/var/lib/postgresql/data
    environment:
      - POSTGRES_DB=postgres
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres

  web:  # Build the app using the Dockerfile above
    build: .
    command: python manage.py runserver 0.0.0.0:8000
    volumes:  # Mount local code so changes reflect inside the container
      - ./code
    ports:  # Map container port to your machine
      - "8000:8000"
    environment:
      - POSTGRES_NAME=postgres
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres
    depends_on:  # Ensure the database starts before the web app
      - db
```

That's all Folks!

